Ticket Booking System

Complete logic documentation: every class and method explained

Stack	Java 21, Spring Boot 4.0.9, Spring Data JPA (Hibernate), PostgreSQL, Spring Security with JWT, ModelMapper, Lombok
Base URL	`/api/v1/...` (REST, JSON)
Scope	All 69 source files: 1 main class, 2 config classes, 3 security classes, 9 entities, 5 enums, 17 DTOs, 9 repositories, 10 services, 10 controllers, 1 scheduler, 2 exception classes

How to read this document: each method has a **Purpose**, a numbered **step by step** explanation of the logic, the **failure cases** (what exception is thrown and which HTTP status the client sees), and **notes** about design choices. A short list of weak points found while reading the code is at the end (section 16).

Contents

1. Project Overview 4

1.1 Technology stack . . . 4
1.2 Layered architecture . . . 4
1.3 Life of a request . . . 5
1.4 Concepts used throughout the code . . . 5
1.5 Package and file map . . . 6

2. Data Model (Entities and Enums) 7

2.1 Entities . . . 7
2.2 Enums (all stored as text, EnumType.STRING) . . . 7
2.3 How the tables connect . . . 8

3. Application Setup and Configuration 9

3.1 TicketBookingSystemApplication . . . 9
3.2 application.properties . . . 9
3.3 ModelMapperConfiguration . . . 9
3.4 SecurityConfiguration . . . 9

4. Security Package (JWT) 12

4.1 JwtService . . . 12
4.2 JwtAuthenticationFilter . . . 13
4.3 CustomUserDetailsService . . . 13

5. Authentication Module 15

5.1 AuthController (/api/v1/auth) . . . 15
5.2 AuthService . . . 15

6. User Module 17

6.1 UserController (/api/v1/users) . . . 17
6.2 UserService . . . 17

7. Movie Module 19

7.1 MovieController (/api/v1/movies) . . . 19
7.2 MovieService . . . 20

8. Theatre, Screen and Seat Modules 22

8.1 TheatreController (/api/v1/theatres) . . . 22
8.2 TheatreService . . . 22
8.3 ScreenController (/api/v1/theatres/{theatreId}/screens) . . . 22

8.4 ScreenService . . . 23
8.5 SeatController (/api/v1/screens) . . . 23
8.6 SeatService . . . 24

9. Show and ShowSeat Modules **25**

9.1 ShowController (/api/v1/shows) . . . 25
9.2 ShowService . . . 25
9.3 ShowSeatController and ShowSeatService . . . 26

10. Booking Module **28**

10.1 BookingController (/api/v1/bookings) . . . 28
10.2 BookingService . . . 28
10.3 BookingSeatController and BookingSeatService . . . 29
10.4 BookingScheduler . . . 29

11. Repository Layer **31**

11.1 Derived queries (SQL generated from the method name) . . . 31
11.2 Custom JPQL queries . . . 31

12. DTO Layer and Validation Rules **33**

12.1 Request DTOs and their validation . . . 33
12.2 Response DTOs . . . 33

13. Exception Handling **34**

13.1 ResourceNotFoundException . . . 34
13.2 GlobalExceptionHandler . . . 34

14. Complete Endpoint Reference **35**

15. End-to-End Walkthrough **36**

16. Observations: Weak Points Found While Reading the Code **37**

1. Project Overview

This is a **movie ticket booking backend**. Administrators build the catalogue (movies, theatres, screens, seats, shows). Customers register, log in, browse shows, look at the seat map of a show and book a seat. A background job later marks bookings as finished once the movie has ended.

1.1 Technology stack

Technology	Role in this project
Spring Boot 4.0.9 / Java 21	Application framework; `@SpringBootApplication` starts the embedded web server and wires all beans.
Spring Web MVC	Controllers (`@RestController`) turn HTTP requests into Java method calls and return JSON.
Spring Data JPA + Hibernate	Repositories (interfaces) generate SQL; entities map to tables. `ddl-auto=update` creates/alters tables automatically.
PostgreSQL	The relational database (`ticket_booking_system` on localhost:5432).
Spring Security + jjwt 0.11.5	Stateless authentication: user logs in, receives a JWT, sends it as `Authorization: Bearer <token>`.
ModelMapper 3.1.1	Copies fields between DTOs and entities so the code does not need manual getters/setters everywhere.
Lombok	Generates getters, setters, constructors at compile time (`@Data`, `@Getter`, `@Setter`, `@NoArgsConstructor`, `@AllArgsConstructor`, `@RequiredArgsConstructor`).
Bean Validation	Annotations such as `@NotBlank`, `@Email`, `@Positive` on DTOs, triggered by `@Valid` in controllers.

1.2 Layered architecture

The project follows the classic controller - service - repository layering. Each layer only talks to the layer directly below it.

Layer	Package	Responsibility
Controller	`controller`	Defines URLs, reads path/query/body, triggers validation, calls one service method, wraps the result in `ResponseEntity` with an HTTP status.
Service	`service`	Business rules (checks, calculations, state changes), transaction boundaries, entity <-> DTO conversion.
Repository	`repository`	Spring Data interfaces; derived queries (`findByEmail`) and custom JPQL (`@Query`).
Entity	`entity`, `enums`	Classes mapped to database tables, and the enumerations stored in them.
DTO	`dto`	Request objects (what clients send, with validation) and response objects (what clients receive). Entities are never exposed, with one exception (ShowSeatController).
Security	`security`, `configuration`	JWT creation/validation, the request filter, the filter chain, password encoder.
Cross-cutting	`exception`, `scheduler`	One central error translator and one periodic background job.

1.3 Life of a request

1. The HTTP request reaches the server. Spring Security's filter chain runs first; our **JwtAuthenticationFilter** looks for `Authorization: Bearer ...`, validates the token and, if valid, stores the logged-in user in the `SecurityContext`.
2. `DispatcherServlet` picks the controller method that matches the URL and HTTP verb.
3. Path variables, query parameters and the JSON body are converted into Java objects. If the parameter has `@Valid`, Bean Validation runs; failures stop here with `MethodArgumentNotValidException`.
4. The controller calls exactly one service method.
5. The service loads entities through repositories, applies business rules, saves changes (inside a transaction when `@Transactional` is present) and converts the result to a response DTO with ModelMapper.
6. The controller wraps the DTO in a `ResponseEntity` (200 OK or 201 CREATED) and Spring serialises it to JSON.
7. If any exception was thrown on the way, **GlobalExceptionHandler** converts it into a 400 / 401 / 404 response.

1.4 Concepts used throughout the code

Concept	Explanation
Constructor injection via `@RequiredArgsConstructor`	Every class declares `private final` dependencies (repositories, ModelMapper...). Lombok generates a constructor taking all `final` fields and Spring uses it to inject the beans. No `@Autowired` needed.
`modelMapper.map(source, Target.class)`	Creates a new Target object and copies every property whose name (or nested path, e.g. `movie.title` -> `movieTitle`) matches the source.
`modelMapper.map(source, existingObject)`	Copies matching properties **into an existing object**. Used for updates. Null values in the source overwrite the target (ModelMapper's default).
`orElseThrow(() -> new ResourceNotFoundException(...))`	`findById` returns an `Optional`. If empty, the lambda creates the exception, which becomes HTTP 404.
Lazy associations (`FetchType.LAZY`)	Related entities (e.g. `booking.getShow()`) are loaded only when first accessed, not with the parent row.
`@GeneratedValue(strategy = SEQUENCE)`	Primary keys come from a database sequence (PostgreSQL), not auto-increment columns.

1.5 Package and file map

Package	Files
root	`TicketBookingSystemApplication`
configuration	`SecurityConfiguration`, `ModelMapperConfiguration`
security	`JwtService`, `JwtAuthenticationFilter`, `CustomUserDetailsService`
entity	`User`, `Movie`, `Theatre`, `Screen`, `Seat`, `Show`, `ShowSeat`, `Booking`, `BookingSeat`
enums	`BookingStatus`, `MovieStatus`, `SeatStatus`, `SeatType`, `ShowStatus`
repository	`UserRepository`, `MovieRepository`, `TheatreRepository`, `ScreenRepository`, `SeatRepository`, `ShowRepository`, `ShowSeatRepository`, `BookingRepository`, `BookingSeatRepository`
service	`AuthService`, `UserService`, `MovieService`, `TheatreService`, `ScreenService`, `SeatService`, `ShowService`, `ShowSeatService`, `BookingService`, `BookingSeatService`
controller	`AuthController`, `UserController`, `MovieController`, `TheatreController`, `ScreenController`, `SeatController`, `ShowController`, `ShowSeatController`, `BookingController`, `BookingSeatController`
dto	17 request/response classes (section 12)
exception	`GlobalExceptionHandler`, `ResourceNotFoundException`
scheduler	`BookingScheduler`

2. Data Model (Entities and Enums)

Understanding the tables is the key to understanding every method, because almost all logic is about creating rows and linking them. The most important design idea is the split between **Seat** (a physical chair in a screen, created once) and **ShowSeat** (that chair for one particular show, with its own price and availability).

2.1 Entities

Entity (table)	Columns / fields	Relationships
User (`users`)	id, name, email (unique), password (BCrypt hash), role ("CUSTOMER"/"ADMIN"), createdAt (`@CreationTimestamp`, set automatically on insert)	none
Movie (`movies`)	id, title, description, genre, language, duration (minutes), releaseDate, certificate, status (MovieStatus)	none
Theatre (`theatres`)	id, name, address, location	none
Screen (`screens`)	id, name (column `screen_name`), totalSeats (`total_seats`)	Many Screens belong to one **Theatre** (`theatre_id`)
Seat (`seats`)	id, seatNumber (`seat_number`), seatType (REGULAR/PREMIUM)	Many Seats belong to one **Screen** (`screen_id`)
Show (`shows`)	id, showDate, startTime, endTime, ticketPrice, status (ShowStatus)	Belongs to one **Movie** (`movie_id`) and one **Screen** (`screen_id`)
ShowSeat (`show_seats`)	id, status (AVAILABLE/BOOKED), price	Belongs to one **Show** (`show_id`) and one **Seat** (`seat_id`)
Booking (`bookings`)	id, totalAmount, bookingDateTime, status (BookingStatus), bookingReference (unique, e.g. `BK-1A2B3C4D`)	Belongs to one **User** as `customer` (`customer_id`) and one **Show** (`show_id`)
BookingSeat (`booking_seats`)	id, priceAtBooking	Belongs to one **Booking** (`booking_id`) and one **ShowSeat** (`show_seat_id`)

2.2 Enums (all stored as text, EnumType.STRING)

Enum	Values	Where it is used
BookingStatus	CONFIRMED, CANCELLED, COMPLETED	Set to CONFIRMED in `createBooking`; changed to COMPLETED by the scheduler. CANCELLED is defined but no code sets it.
MovieStatus	ACTIVE, INACTIVE	ACTIVE on creation; `addShow` refuses non-ACTIVE movies. No API sets INACTIVE.
SeatStatus	AVAILABLE, BOOKED	ShowSeat state. AVAILABLE on creation, BOOKED after a booking.
SeatType	REGULAR, PREMIUM	Premium seats cost 100 more when show seats are generated.
ShowStatus	SCHEDULED, CANCELLED, COMPLETED	SCHEDULED on creation; booking requires SCHEDULED. The other two are not set by any code.

2.3 How the tables connect

```
Theatre 1 ---- * Screen 1 ---- * Seat
                  |                |
                  | 1              | 1
                  *                *
Movie   1 ---- * Show  1 ---- * ShowSeat * ---- 1 Seat   (ShowSeat = Seat for one Show)
                  | 1               | 1
                  *                 *
User    1 ---- * Booking 1 ---- * BookingSeat * ---- 1 ShowSeat
```

Reading the diagram: a Theatre has many Screens, a Screen has many Seats. A Show puts a Movie on a Screen at a date/time. When a Show is created, one ShowSeat is generated for every Seat of that Screen. A Booking is made by a User for a Show, and each BookingSeat links the Booking to the ShowSeat that was reserved, remembering the price paid.

Lombok on entities: `@Data` generates getters, setters, `equals`, `hashCode`, `toString`; `@NoArgsConstructor` is required by JPA; `@AllArgsConstructor` creates a constructor with every field.

3. Application Setup and Configuration

3.1 TicketBookingSystemApplication

main

```
public static void main(String[] args)
```

PURPOSE

Entry point of the whole application.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `SpringApplication.run(...)` boots Spring: it scans the package `com.example.ticket_booking_system` for `@Component`, `@Service`, `@RestController`, `@Configuration` classes and creates them as beans.
2. `@SpringBootApplication` = `@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`. Auto-configuration sees the dependencies in `pom.xml` and sets up the web server, the PostgreSQL DataSource, Hibernate and Spring Security.
3. `@EnableScheduling` switches on Spring's scheduler so that the `@Scheduled` method in `BookingScheduler` actually runs.

3.2 application.properties

Property	Meaning
`spring.datasource.url/username/password`	JDBC connection to PostgreSQL database `ticket_booking_system` on localhost:5432, user `postgres`.
`spring.jpa.hibernate.ddl-auto=update`	At startup Hibernate compares entities to the schema and creates/adds missing tables and columns (never drops).
`spring.jpa.show-sql=true`, `hibernate.format_sql=true`	Prints every generated SQL statement to the console, nicely formatted. Useful for learning.
`hibernate.dialect=PostgreSQLDialect`	Tells Hibernate to generate PostgreSQL-flavoured SQL.
`spring.security.user.name/password`	Default in-memory user (admin/admin123). Because the project defines its own `UserDetailsService`, these values are effectively unused.
`jwt.secret`	The signing secret read by `JwtService` through `@Value("${jwt.secret}")`.

3.3 ModelMapperConfiguration

modelMapper

```
@Bean public ModelMapper modelMapper()
```

PURPOSE

Registers one shared `ModelMapper` object in the Spring container.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `@Configuration` marks the class as a source of beans; `@Bean` makes Spring call the method once and keep the returned object.
2. Every service that declares `private final ModelMapper modelMapper` receives this same instance through constructor injection.

3.4 SecurityConfiguration

Defines how Spring Security behaves. It contains one filter chain and three helper beans.

securityFilterChain

```
@Bean SecurityFilterChain securityFilterChain(HttpSecurity http, JwtAuthenticationFilter jwtAuthenticationFilter)
```

PURPOSE

Builds the list of security filters and rules applied to every HTTP request.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `csrf.disable()`: CSRF protection is meant for cookie/session logins. This API uses tokens in a header, so it is switched off.
2. `sessionCreationPolicy(STATELESS)`: the server never creates an HTTP session. Every request must carry its own token, so authentication is re-derived from the JWT each time.
3. `authorizeHttpRequests(auth -> auth.anyRequest().permitAll())`: **currently every URL is open to everybody**, logged in or not.
4. The long commented-out block shows the intended rules: register/login/Swagger/GET browsing public; user listing, movie/theatre/show management ADMIN only; bookings and `/users/me` for CUSTOMER or ADMIN; anything else authenticated. Re-enabling it replaces the `permitAll` line.
5. `addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class)`: inserts our JWT filter before Spring's username/password filter, so the SecurityContext is already filled when the request reaches the controllers.
6. `http.build()` creates and returns the finished chain.

NOTES

- Because of `permitAll`, the JWT is only used to *identify* the caller (e.g. `/users/me`), not to *restrict* access.

authenticationManager

```
@Bean AuthenticationManager authenticationManager(AuthenticationConfiguration configuration)
```

PURPOSE

Exposes Spring's `AuthenticationManager` as a bean so `AuthService.login` can inject it.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `configuration.getAuthenticationManager()` returns the manager that Spring built from the registered `AuthenticationProvider` beans (here the `DaoAuthenticationProvider` below).
2. `AuthService.login` calls `authenticate(...)` on it to check email and password.

passwordEncoder

```
@Bean PasswordEncoder passwordEncoder()
```

PURPOSE

Provides the password hashing algorithm.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Returns `new BCryptPasswordEncoder()`. BCrypt is a one-way, salted hash: the same password produces a different hash each time, and the original cannot be recovered.
2. Used in two places: `AuthService.register` calls `encode(rawPassword)` before saving; the `DaoAuthenticationProvider` calls `matches(raw, hash)` during login.

authenticationProvider

```
@Bean AuthenticationProvider authenticationProvider(PasswordEncoder passwordEncoder, CustomUserDetailsService userDetailsService)
```

PURPOSE

Tells Spring Security how to verify a login: load the user from our database and compare the password hash.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Creates a `DaoAuthenticationProvider` with our `CustomUserDetailsService` (the user source).
2. Sets the BCrypt encoder as its password checker.
3. At login the provider calls `loadUserByUsername(email)`, then `passwordEncoder.matches(typedPassword, storedHash)`. A mismatch throws `BadCredentialsException`.

4. Security Package (JWT)

JWT (JSON Web Token) is a signed string with three parts: header, payload (claims such as subject and expiry), and signature. The server signs the token with a secret key at login; later it verifies the signature, so it can trust the claims without a database lookup or a session.

4.1 JwtService

Creates and verifies tokens. The secret is read from `jwt.secret` in the constructor and turned into an HMAC key.

JwtService (constructor)

```
public JwtService(@Value("${jwt.secret}") String key)
```

PURPOSE

Prepares the signing key once when the bean is created.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Spring injects the text of `jwt.secret` into `key`.
2. `Keys.hmacShaKeyFor(key.getBytes())` converts the bytes into a `SecretKey` suitable for HMAC-SHA signing. jjwt chooses HS256/HS384/HS512 based on key length (the configured secret is long enough for HS512).
3. The key is stored in the final field `SECRET_KEY` and reused for every sign/verify operation.

generateToken

```
public String generateToken(String username)
```

PURPOSE

Produces a signed JWT for a user (the "username" is the email).

HOW THE LOGIC WORKS (STEP BY STEP)

1. `Jwts.builder()` starts a new token.
2. `setSubject(username)`: the email is stored as the `sub` claim, which is how the user is identified later.
3. `setIssuedAt(now)`: stores the creation time.
4. `setExpiration(now + EXP_TIME)`: stores the time after which the token is invalid.
5. `signWith(SECRET_KEY)`: computes the signature over header+payload.
6. `compact()` returns the final `xxxxx.yyyyy.zzzzz` string.

NOTES

- `EXP_TIME = 60*60*24` equals 86,400, but `Date` arithmetic uses **milliseconds**. So a token lives about **86 seconds**, not one day as the comment says. See section 16 for the fix (`* 1000`).

getUserName

```
public String getUserName(String token)
```

PURPOSE

Reads the email out of a token.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `Jwts.parserBuilder().setSigningKey(SECRET_KEY).build()` creates a parser that knows the key.
2. `parseClaimsJws(token)` checks the signature and expiry. If the token was tampered with or expired it throws an exception.
3. `getBody().getSubject()` returns the `sub` claim, i.e. the email.

validateToken

```
public boolean validateToken(String token)
```

PURPOSE

Answers "is this token genuine and still valid?" without throwing.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Runs the same parse as `getUserName`.
2. If parsing succeeds, the signature is correct and the token is not expired: return `true`.
3. Any exception (bad signature, malformed string, expired) is caught and the method returns `false`.

4.2 JwtAuthenticationFilter

Extends `OncePerRequestFilter`, which guarantees the filter runs exactly once per request. It is a `@Component` so Spring creates it and injects `JwtService` and `CustomUserDetailsService`.

doFilterInternal

```
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
FilterChain filterChain)
```

PURPOSE

Looks at every incoming request and, when it carries a valid token, marks the request as authenticated.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Reads the `Authorization` header.
2. If the header is missing or does not start with `"Bearer "`, nothing happens and the request continues anonymously.
3. Otherwise `substring(7)` removes the 7 characters `"Bearer "`, leaving the raw token.
4. `jwtService.validateToken(token)`: if false (bad/expired), the filter again does nothing special; the request continues unauthenticated.
5. If valid: `getUserName(token)` extracts the email and `customUserDetailsService.loadUserByUsername(email)` loads the user (with roles) from the database.
6. Creates `UsernamePasswordAuthenticationToken(userDetails, null, authorities)`. The `null` is the credentials: the password is not needed because the token already proved identity.
7. `SecurityContextHolder.getContext().setAuthentication(auth)` stores it for the current request thread. From now on controllers can receive an `Authentication` parameter whose `getName()` is the email.
8. `filterChain.doFilter(request, response)` passes the request on, always, whether or not authentication happened.

NOTES

- This filter never rejects a request itself. Rejection is the job of the authorization rules in `SecurityConfiguration` (currently `permitAll`).
- If a valid token belongs to a user that no longer exists, `loadUserByUsername` throws inside the filter, where `GlobalExceptionHandler` cannot catch it.

4.3 CustomUserDetailsService

loadUserByUsername

```
public UserDetails loadUserByUsername(String email)
```

PURPOSE

Bridges our `User` table to Spring Security's `UserDetails` object. Called by the login provider and by the JWT filter.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `userRepository.findByEmail(email)` loads the user.

2. If absent, throws `ResourceNotFoundException("User not found with email: ...")`.
3. Builds a Spring `User` with `withUsername(email)`, `password(hashFromDb)`, `roles(user.getRole())`, `build()`.
4. `roles("CUSTOMER")` automatically adds the prefix, producing the authority `ROLE_CUSTOMER`. That prefix is why the commented rules use `hasRole("ADMIN")`.

FAILURE CASES / HTTP RESULT

- During login, an unknown email makes `DaoAuthenticationProvider` wrap the exception in an `InternalAuthenticationServiceException` (an `AuthenticationException`), so the client typically gets **401** with the message.

NOTES

- The class name `org.springframework.security.core.userdetails.User` is written fully qualified because our own entity is also named `User`.

5. Authentication Module

5.1 AuthController (/api/v1/auth)

register

```
public ResponseEntity<UserResponseDTO> register(@Valid @RequestBody RegisterRequestDTO requestDTO)
```

Endpoint: `POST /api/v1/auth/register`

PURPOSE

Public endpoint for creating a customer account.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `@RequestBody` converts the JSON into a `RegisterRequestDTO`.
2. `@Valid` runs validation: name not blank, email valid, password at least 8 characters.
3. Calls `authService.register(requestDTO)`.
4. Returns the created user (no password) with status **201 CREATED**.

FAILURE CASES / HTTP RESULT

- Validation fails: 400 with a map `{field: message}`.
- Email already used: 400 (from the service).

login

```
public ResponseEntity<AuthResponseDTO> login(@Valid @RequestBody LoginRequestDTO requestDTO)
```

Endpoint: `POST /api/v1/auth/login`

PURPOSE

Public endpoint that exchanges email+password for a JWT.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Validates the body (email format, both fields not blank).
2. Calls `authService.login(requestDTO)`.
3. Returns `{token, tokenType}` with **200 OK**.

FAILURE CASES / HTTP RESULT

- Wrong password: `BadCredentialsException` -> 401.
- Unknown email: 401 (see section 4.3).

5.2 AuthService

register

```
public UserResponseDTO register(RegisterRequestDTO requestDTO)
```

PURPOSE

Creates a new CUSTOMER account safely.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `userRepository.existsByEmail(email)` (SQL `select count > 0 ... where email = ?`). If true, throw `IllegalArgumentException("Email is already registered")`.
2. `modelMapper.map(requestDTO, User.class)` builds a `User` and copies `name`, `email`, `password` from the DTO.
3. `passwordEncoder.encode(...)` replaces the plain password with its BCrypt hash. The plain text is never stored.
4. `user.setRole("CUSTOMER")`: the role is forced by the server, so a client cannot register itself as ADMIN.
5. `userRepository.save(user)` inserts the row (`createdAt` is filled by `@CreationTimestamp`; the `email` unique constraint is a second line of defence).

6. Maps the saved entity to `UserResponseDTO` (id, name, email, role) so the hash is not returned.

FAILURE CASES / HTTP RESULT

- Duplicate email -> `IllegalArgumentException` -> **400**.

NOTES

- There is no API for creating an ADMIN. An admin user must be inserted/updated directly in the database.

login

```
public AuthResponseDTO login(LoginRequestDTO requestDTO)
```

PURPOSE

Verifies credentials and returns a token.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `authenticationManager.authenticate(new UsernamePasswordAuthenticationToken(email, password))` hands the typed credentials to the `DaoAuthenticationProvider`.
2. The provider loads the user (`CustomUserDetailsService`), BCrypt-compares the password with the stored hash. On mismatch it throws an `AuthenticationException` and the method stops.
3. On success, `jwtService.generateToken(email)` creates the JWT.
4. Returns `new AuthResponseDTO(token, "Bearer")`. The client then sends `Authorization: Bearer <token>` on later requests.

FAILURE CASES / HTTP RESULT

- Bad credentials -> `AuthenticationException` -> **401** (handled by `GlobalExceptionHandler`).

6. User Module

6.1 UserController (/api/v1/users)

getMyProfile

```
public ResponseEntity<UserResponseDTO> getMyProfile(Authentication authentication)
```

Endpoint: `GET /api/v1/users/me`

PURPOSE

Returns the profile of whoever is logged in.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Spring injects the `Authentication` object created by `JwtAuthenticationFilter` for this request.
2. `authentication.getName()` is the email stored in the token.
3. Calls `userService.getMyProfile(email)` and returns it with **200 OK**.
4. Route `/me` is a fixed path, so it does not clash with other user routes.

FAILURE CASES / HTTP RESULT

- No/invalid token: `authentication` is null, which causes a NullPointerException (500) because `permitAll` lets the request in.

updateMyProfile

```
public ResponseEntity<UserResponseDTO> updateMyProfile(Authentication authentication, @Valid
@RequestBody UserUpdateRequestDTO requestDTO)
```

Endpoint: `PUT /api/v1/users/me`

PURPOSE

Lets the logged-in user change their own name and email.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Validates the body (`name` required, `email` required and well formed, with custom messages).
2. Takes the email from `authentication.getName()` so a user can only edit themselves.
3. Calls `userService.updateMyProfile(email, dto)` and returns 200.

getAllUsers

```
public ResponseEntity<Page<UserResponseDTO>> getAllUsers(Pageable pageable)
```

Endpoint: `GET /api/v1/users?page=&size=&sort=`

PURPOSE

Admin-style listing of all users, paginated.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Spring builds a `Pageable` from the query string: `?page=0&size=10&sort=name,asc`.
2. Passes it to `userService.getAllUsers`.
3. Returns a `Page` object: the content list plus metadata (total elements, total pages, page number...).

NOTES

- Intended to be ADMIN only (see the commented rules); currently open.

6.2 UserService

getMyProfile

```
public UserResponseDTO getMyProfile(String email)
```

PURPOSE

Loads one user by email and converts it to a DTO.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `findByEmail(email)` returns an `Optional<User>`.
2. Empty -> `ResourceNotFoundException` (404).
3. `modelMapper.map(user, UserResponseDTO.class)` copies id, name, email, role. The password hash is not in the DTO, so it is never exposed.

updateMyProfile

```
public UserResponseDTO updateMyProfile(String email, UserUpdateRequestDTO requestDTO)
```

PURPOSE

Updates name and email of an existing user.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Loads the user by the current email or throws 404.
2. `modelMapper.map(requestDTO, user)` overwrites `name` and `email` **on the same managed entity** (password, role, id are untouched because the DTO has no such fields).
3. `save(user)` issues an UPDATE.
4. Returns the updated user as DTO.

NOTES

- If the new email already belongs to another user, the database unique constraint fails (no friendly check in code).
- Changing the email changes the identity stored in the JWT subject. The old token will then fail to find a user.

getAllUsers

```
public Page<UserResponseDTO> getAllUsers(Pageable pageable)
```

PURPOSE

Returns one page of users as DTOs.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `userRepository.findAll(pageable)` runs `select ... limit ? offset ?` plus a count query and returns `Page<User>`.
2. `Page.map(fn)` transforms each element, keeping the paging metadata, so every `User` becomes a `UserResponseDTO`.

7. Movie Module

7.1 MovieController (/api/v1/movies)

The class has `@Validated`, which enables validation of simple parameters such as `@PathVariable @Positive Long id` (id must be greater than 0).

createMovie

```
public ResponseEntity<MovieResponseDTO> createMovie(@Valid @RequestBody MovieRequestDTO dto)
```

Endpoint: `POST /api/v1/movies`

PURPOSE

Adds a movie to the catalogue.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Validation: title, genre, language, certificate not blank; duration positive; releaseDate not null.
2. Calls the service; returns **201 CREATED** with the saved movie.

getAllMovies

```
public ResponseEntity<List<MovieResponseDTO>> getAllMovies()
```

Endpoint: `GET /api/v1/movies`

PURPOSE

Lists every movie.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Calls `movieService.getAllMovies()`; returns 200.

getMovieById

```
public ResponseEntity<MovieResponseDTO> getMovieById(@PathVariable @Positive Long id)
```

Endpoint: `GET /api/v1/movies/{id}`

PURPOSE

Fetches one movie.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `@Positive` rejects 0 or negative ids.
2. Calls the service; returns 200.

FAILURE CASES / HTTP RESULT

- Unknown id: 404.

updateMovieById

```
public ResponseEntity<MovieResponseDTO> updateMovieById(@PathVariable @Positive Long id, @Valid
@RequestBody MovieRequestDTO dto)
```

Endpoint: `PUT /api/v1/movies/{id}`

PURPOSE

Replaces the editable fields of an existing movie.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Validates id and body, delegates to the service, returns 200.

getAllMoviesByTitle

```
public ResponseEntity<List<MovieResponseDTO>> getAllMoviesByTitle(@RequestParam String title)
```

Endpoint: `GET /api/v1/movies/title?title=`

PURPOSE

Searches movies by exact title, ignoring upper/lower case.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `@RequestParam` reads `?title=Leo`; the service does the search; returns 200 (empty list if none).

NOTES

- `/title` and `/genre` are literal paths. Spring prefers a literal match over the pattern `/{id}`, so there is no conflict.

getAllMoviesByGenre

```
public ResponseEntity<List<MovieResponseDTO>> getAllMoviesByGenre(@RequestParam String genre)
```

Endpoint: `GET /api/v1/movies/genre?genre=`

PURPOSE

Searches movies by genre, case-insensitive.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Same flow as the title search using `?genre=`.

7.2 MovieService

createMovie

```
public MovieResponseDTO createMovie(MovieRequestDTO movieRequestDTO)
```

PURPOSE

Creates and stores a movie.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `modelMapper.map(dto, Movie.class)` builds a `Movie` with title, description, genre, language, duration, releaseDate, certificate.
2. `setStatus(ACTIVE)`: new movies are always active (clients cannot choose).
3. `save` inserts the row and returns it with the generated id.
4. Converts to `MovieResponseDTO`.

getAllMovies

```
public List<MovieResponseDTO> getAllMovies()
```

PURPOSE

Returns all movies.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `findAll()` returns every `Movie`.
2. A stream converts each entity with `modelMapper.map(movie, MovieResponseDTO.class)`.
3. `.toList()` gives the final immutable list. (The commented-out loop is the same logic written without streams.)

getMovieById

```
public MovieResponseDTO getMovieById(Long id)
```

PURPOSE

Returns one movie or a 404.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `findById(id).orElseThrow(...)` throws `ResourceNotFoundException("Movie id X is not found")`.
2. Maps the entity to a DTO and returns it.

updateMovieById

```
public MovieResponseDTO updateMovieById(Long id, MovieRequestDTO dto)
```

PURPOSE

Edits an existing movie.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Loads the movie or throws 404.
2. `modelMapper.map(dto, movie)` copies all seven editable fields onto the loaded entity. `id` and `status` are not in the DTO, so they stay unchanged.
3. `save` runs an UPDATE; returns the result as DTO.

NOTES

- Because ModelMapper copies nulls, sending `description: null` clears the description. There is no way through this API to set a movie INACTIVE.

getAllMoviesByTitle

```
public List<MovieResponseDTO> getAllMoviesByTitle(String title)
```

PURPOSE

Finds movies whose title equals the text, ignoring case.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `findByTitleIgnoreCase(title)`: Spring Data derives `where upper(title) = upper(?)`.
2. A `for` loop maps each result to a DTO and adds it to a new list; the list is returned.

getAllMoviesByGenre

```
public List<MovieResponseDTO> getAllMoviesByGenre(String genre)
```

PURPOSE

Finds movies of one genre, ignoring case.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `findByGenreIgnoreCase(genre)` runs the derived query.
2. Maps each movie to a DTO in a loop and returns the list.

8. Theatre, Screen and Seat Modules

These three modules build the physical venue. The order matters: a Theatre must exist before a Screen can be added to it, and a Screen must exist before Seats can be added.

8.1 TheatreController (/api/v1/theatres)

addTheatre

```
public ResponseEntity<TheatreResponseDTO> addTheatre(@Valid @RequestBody TheatreRequestDTO dto)
```

Endpoint: `POST /api/v1/theatres`

PURPOSE

Creates a theatre.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Validates name, address, location (all not blank).
2. Calls the service; returns **201**.

getAllTheatres

```
public ResponseEntity<List<TheatreResponseDTO>> getAllTheatres()
```

Endpoint: `GET /api/v1/theatres`

PURPOSE

Lists all theatres.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Delegates to the service; returns 200.

8.2 TheatreService

addTheatre

```
public TheatreResponseDTO addTheatre(TheatreRequestDTO requestDTO)
```

PURPOSE

Stores a new theatre.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Maps DTO to `Theatre` with ModelMapper.
2. `save` inserts and returns the entity with its id.
3. Maps the saved entity to `TheatreResponseDTO`.

getAllTheatres

```
public List<TheatreResponseDTO> getAllTheatres()
```

PURPOSE

Returns all theatres.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `findAll()` loads every theatre.
2. Loops over them, mapping each to a DTO and adding it to a new `ArrayList`.
3. Returns the list.

8.3 ScreenController (/api/v1/theatres/{theatreId}/screens)

addScreen

```
public ResponseEntity<ScreenResponseDTO> addScreen(@PathVariable Long theatreId, @Valid
@RequestBody ScreenRequestDTO dto)
```

Endpoint: `POST /api/v1/theatres/{theatreId}/screens`

PURPOSE

Adds a screen (auditorium) inside a theatre.

HOW THE LOGIC WORKS (STEP BY STEP)

1. The theatre id comes from the URL, making the parent-child relationship explicit (REST nesting).
2. Validates `name` not blank and `totalSeats` not null and positive.
3. Calls the service; returns **201**.

8.4 ScreenService

addScreen

```
public ScreenResponseDTO addScreen(Long theatreId, ScreenRequestDTO requestDTO)
```

PURPOSE

Creates a screen linked to its theatre.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `theatreRepository.findById(theatreId)` or throw `ResourceNotFoundException("Theatre not found")` (404).
2. `modelMapper.map(dto, Screen.class)` copies `name` and `totalSeats`.
3. `screen.setTheatre(theatre)` sets the foreign key `theatre_id`.
4. `save` inserts the row.
5. Maps to `ScreenResponseDTO`; ModelMapper reads `theatre.id` and `theatre.name` into `theatreId` and `theatreName`.

NOTES

- `totalSeats` is just a number stored on the screen. It is **not** compared with the number of `Seat` rows actually created.

8.5 SeatController (/api/v1/screens)

addSeat

```
public ResponseEntity<SeatResponseDTO> addSeat(@PathVariable @Positive Long screenId, @Valid
@RequestBody SeatRequestDTO dto)
```

Endpoint: `POST /api/v1/screens/{screenId}/seats`

PURPOSE

Adds one physical seat to a screen.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Validates `screenId > 0`, `seatNumber` not blank, `seatType` not null (REGULAR or PREMIUM).
2. Returns the created seat with **201**.

getSeatsByScreenId

```
public ResponseEntity<List<SeatResponseDTO>> getSeatsByScreenId(@PathVariable @Positive Long screenId)
```

Endpoint: `GET /api/v1/screens/{screenId}/seats`

PURPOSE

Lists all seats of a screen.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Validates the id; returns 200 with the list.

8.6 SeatService

addSeat

```
public SeatResponseDTO addSeat(Long screenId, SeatRequestDTO requestDTO)
```

PURPOSE

Creates a seat while preventing duplicate seat numbers on one screen.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Load the screen or throw 404 (`Screen not found with ID: ...`).
2. `existsByScreenIdAndSeatNumber(screenId, seatNumber)` -> SQL `select count>0 from seats where screen_id=? and seat_number=?`. If true throw `IllegalArgumentException("Seat number already exists in this screen")`.
3. Map DTO to `Seat` (seatNumber, seatType), attach the screen, `save`.
4. Map to `SeatResponseDTO` and set `screenId` and `screenName` manually from the screen.

FAILURE CASES / HTTP RESULT

- Screen missing: 404.
- Duplicate seat number: 400.

NOTES

- The same seat number (e.g. "A1") may exist on different screens, only the combination screen+number must be unique.

getSeatsByScreenId

```
public List<SeatResponseDTO> getSeatsByScreenId(Long screenId)
```

PURPOSE

Returns every seat of the given screen.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `screenRepository.findById(screenId)` is called only to **verify the screen exists** (404 otherwise); the result is not stored.
2. `seatRepository.findByScreenId(screenId)` loads the seats (derived query on the `screen.id` path).
3. For each seat: map to DTO and set `screenId` and `screenName` from `seat.getScreen()` (lazy association, loaded on access).
4. Return the list.

9. Show and ShowSeat Modules

A **Show** is one screening: a Movie on a Screen at a date and time with a base ticket price. Creating a show also generates its seat map (ShowSeat rows). This is the most important admin operation because bookings depend on it.

9.1 ShowController (/api/v1/shows)

addShow

```
public ResponseEntity<ShowResponseDTO> addShow(@PathVariable Long movieId, @PathVariable Long screenId, @Valid @RequestBody ShowRequestDTO dto)
```

Endpoint: `POST /api/v1/shows/{movieId}/{screenId}`

PURPOSE

Schedules a movie on a screen.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Movie id and screen id come from the URL; the body holds date, start time, end time, ticket price.
2. Validation: all four fields not null; `showDate` must be today or in the future (`@FutureOrPresent`).
3. Delegates to the service and returns **201** with the created show.

getAllShows

```
public ResponseEntity<List<ShowResponseDTO>> getAllShows(@RequestParam(required=false) Long movieId, @RequestParam(required=false) Long theatreId, @RequestParam(required=false) LocalDate showDate)
```

Endpoint: `GET /api/v1/shows?movieId=&theatreId=&showDate=`

PURPOSE

Searches shows with optional filters.

HOW THE LOGIC WORKS (STEP BY STEP)

1. All three parameters are optional (`required = false`); missing ones arrive as `null`.
2. `showDate` is parsed from ISO text like `2026-11-20`.
3. Passes them to the service; returns 200.

getShowById

```
public ResponseEntity<ShowResponseDTO> getShowById(@PathVariable @Positive Long id)
```

Endpoint: `GET /api/v1/shows/{id}`

PURPOSE

Fetches one show.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Validates id, delegates, returns 200 (404 if unknown).

9.2 ShowService

addShow

```
public ShowResponseDTO addShow(Long movieId, Long screenId, ShowRequestDTO requestDTO)
```

PURPOSE

Creates a show after four business checks and generates the seat map for it.

HOW THE LOGIC WORKS (STEP BY STEP)

1. **Time check:** `endTime.isAfter(startTime)` must be true, otherwise `IllegalArgumentException("End time must be after start time")`.
2. **Movie check:** load the movie or 404. If `status != ACTIVE`, throw `IllegalArgumentException("Cannot schedule a show for an inactive movie")`.
3. **Screen check:** load the screen or 404.
4. **Clash check:** `showRepository.existsOverlappingShow(screenId, date, start, end)`. Two time ranges overlap when `existing.start < new.end` AND `existing.end > new.start`. Example: existing 10:00-12:00, new 11:00-13:00 -> 10<13 and 12>11 -> overlap -> error. New 12:00-14:00 -> 12>12 is false -> allowed (back-to-back is fine).
5. **Create the show:** map the DTO to `Show` (date, start, end, ticketPrice), set the movie and screen, set status `SCHEDULED`, `save`.
6. **Load physical seats:** `seatRepository.findByScreenId(screenId)`.
7. **Generate ShowSeats:** for every `Seat` create a `ShowSeat` linked to the saved show and the seat, status `AVAILABLE`, and price = `ticketPrice + 100` if the seat is PREMIUM, otherwise `ticketPrice`.
8. `showSeatRepository.saveAll(showSeats)` inserts all of them (batched).
9. Map the saved show to `ShowResponseDTO` and add `theatreId` and `theatreName` from `screen.getTheatre()`.

FAILURE CASES / HTTP RESULT

- Bad times, inactive movie, overlapping show: **400**.
- Movie or screen not found: **404**.

NOTES

- The +100 premium surcharge is hard-coded.
- The method is not `@Transactional`, so the show and its seats are saved in separate transactions. If seat creation failed, a show with no seats would remain.
- The seat map is a snapshot: seats added to the screen **after** the show is created do not appear for that show.
- The overlap query does not ignore CANCELLED shows.

getAllShows

```
public List<ShowResponseDTO> getAllShows(Long movieId, Long theatreId, LocalDate showDate)
```

PURPOSE

Returns shows matching any combination of filters.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Calls `findShowsByFilters(movieId, theatreId, showDate)`. Each filter is applied only when not null (see section 11).
2. Maps each `Show` to `ShowResponseDTO` in a loop. ModelMapper fills `movieId`, `movieTitle`, `screenId`, `screenName`, `theatreId`, `theatreName` by walking the nested properties.
3. Returns the list.

getShowById

```
public ShowResponseDTO getShowById(Long id)
```

PURPOSE

Returns one show.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `findById(id)` or `ResourceNotFoundException("Show not found with ID: ...")`.
2. Maps to DTO.

9.3 ShowSeatController and ShowSeatService

getShowSeats (controller)

```
public ResponseEntity<List<ShowSeat>> getShowSeats(@PathVariable Long showId)
```

Endpoint: `GET /api/v1/shows/{showId}/seats`

PURPOSE

Returns the seat map of a show: which seats are free or booked and their prices. The frontend uses this to draw the seat selection screen.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Reads the show id from the URL.
2. Calls `showSeatService.getShowSeats(showId)`.
3. Returns the list with 200.

NOTES

- Unlike the other endpoints this returns the **entity** `ShowSeat`, not a DTO, so the JSON includes nested show and seat objects. A DTO would be cleaner.

getShowSeats (service)

```
public List<ShowSeat> getShowSeats(Long showId)
```

PURPOSE

Loads all seat rows of one show.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `showSeatRepository.findByShowId(showId)` -> `select ... from show_seats where show_id = ?`.
2. Returns the entities unchanged. An unknown show id simply returns an empty list (no 404).

10. Booking Module

10.1 BookingController (/api/v1/bookings)

createBooking

```
public ResponseEntity<BookingResponseDTO> createBooking(@PathVariable Long userId,
@PathVariable Long showId, @PathVariable Long showSeatId)
```

Endpoint: `POST /api/v1/bookings/users/{userId}/shows/{showId}/seats/{showSeatId}`

PURPOSE

Books one seat of a show for a user.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Everything needed is encoded in the URL: who (`userId`), which show (`showId`), which seat (`showSeatId`). There is no request body.
2. Calls `bookingService.createBooking(...)`.
3. Returns **201 CREATED** with the booking details.

NOTES

- The user id is taken from the URL rather than from the logged-in user, so any caller could book on behalf of any user.

10.2 BookingService

createBooking

```
@Transactional public BookingResponseDTO createBooking(Long userId, Long showId, Long
showSeatId)
```

PURPOSE

The core business operation: validate everything, then create the booking, reserve the seat and return a receipt.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `@Transactional`: all database changes below succeed together or are rolled back together if any exception occurs.
2. **Load the user** or 404 (`User not found with ID`).
3. **Load the show** or 404.
4. **Show must be SCHEDULED**, otherwise `IllegalArgumentException("This show is not available for booking")`.
5. **Load the show seat** or 404.
6. **Seat must belong to this show:** `showSeat.getShow().getId().equals(showId)`, otherwise `IllegalArgumentException`. This stops someone passing a seat id from another show.
7. **Seat must be AVAILABLE**, otherwise `IllegalArgumentException("The selected seat is not available")`.
8. **Create the Booking:** customer = user, show = show, `totalAmount` = the seat price, `bookingDateTime` = now, status = `CONFIRMED`.
9. **Reference number:** `"BK-" + first 8 characters of a random UUID, upper-case`, e.g. `BK-3F9A21CE`. The column is unique.
10. `bookingRepository.save(booking)` inserts the booking.
11. **Create the BookingSeat:** links the saved booking to the show seat and stores `priceAtBooking`, a snapshot so later price changes do not alter history. `save`.
12. **Reserve the seat:** `showSeat.setStatus(BOOKED)` and `save`. Future attempts fail at the AVAILABLE check.
13. **Build the response:** map the booking to `BookingResponseDTO`, then set `customerId`, `customerName`, `showId` manually.

14. On leaving the method the transaction commits.

FAILURE CASES / HTTP RESULT

- User / show / seat not found: **404**.
- Show not scheduled, seat from another show, seat already booked: **400**.

NOTES

- Only one seat per booking. For several seats the method would be called several times, creating several bookings.
- There is no locking or `@Version` on `ShowSeat`. Two simultaneous requests can both read AVAILABLE and both succeed (double booking). See section 16.
- There is no cancel operation, so a booked seat never returns to AVAILABLE.

10.3 BookingSeatController and BookingSeatService

getBookingSeats (controller)

```
public ResponseEntity<List<BookingSeatResponseDTO>> getBookingSeats(@PathVariable Long bookingId)
```

Endpoint: `GET /api/v1/bookings/{bookingId}/seats`

PURPOSE

Shows which seat(s) a booking contains.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Calls the service and returns 200.

getBookingSeats (service)

```
public List<BookingSeatResponseDTO> getBookingSeats(Long bookingId)
```

PURPOSE

Lists the BookingSeat rows of a booking.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `bookingRepository.existsById(bookingId)`; if false throw `ResourceNotFoundException` (404).
2. `bookingSeatRepository.findByBookingId(bookingId)` loads the rows.
3. For each row: map to DTO, then set `bookingId` from `getBooking().getId()` and `showSeatId` from `getShowSeat().getId()` explicitly.
4. Return the list (id, bookingId, showSeatId, priceAtBooking).

10.4 BookingScheduler

completeBookingsForFinishedShows

```
@Scheduled(fixedRate = 60000) @Transactional public void completeBookingsForFinishedShows()
```

PURPOSE

A background job that automatically changes bookings from CONFIRMED to COMPLETED once the movie has ended.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `@Scheduled(fixedRate = 60000)`: Spring runs the method every 60 seconds (enabled by `@EnableScheduling`).
2. Calls `findBookingsForCompletedShows(CONFIRMED, today, nowTime)`. The query returns CONFIRMED bookings whose show date is before today, **or** whose show is today and `endTime <= now`.
3. For each booking returned: `setStatus(COMPLETED)`.
4. `saveAll(bookings)` writes them. Because the method is `@Transactional` the entities are managed and would also be flushed on commit, so this call is a safety net.

5. If nothing qualifies the list is empty and nothing happens.

NOTES

- Uses the server's local clock and timezone (`LocalDate.now()`, `LocalTime.now()`).

11. Repository Layer

Repositories are interfaces; Spring Data generates the implementation at runtime. All extend `JpaRepository<Entity, Long>`, which already provides `save`, `saveAll`, `findById`, `findAll`, `findAll(Pageable)`, `existsById`, `deleteById`, `count` and more. Only the extra methods are explained here.

11.1 Derived queries (SQL generated from the method name)

Repository.method	How the name is read	Equivalent SQL
`UserRepository.findByEmail`	find + By + Email; returns Optional	`select * from users where email = ?`
`UserRepository.existsByEmail`	exists + By + Email	`select count(*) > 0 from users where email = ?`
`MovieRepository.findByTitleIgnoreCase`	title equals, ignoring case	`... where upper(title) = upper(?)`
`MovieRepository.findByGenreIgnoreCase`	genre equals, ignoring case	`... where upper(genre) = upper(?)`
`SeatRepository.existsByScreenIdAndSeatNumber`	screen.id = ? AND seatNumber = ?	`select count(*) > 0 from seats where screen_id=? and seat_number=?`
`SeatRepository.findByScreenId`	screen.id = ?	`select * from seats where screen_id = ?`
`ShowSeatRepository.findByShowId`	show.id = ?	`select * from show_seats where show_id = ?`
`BookingSeatRepository.findByBookingId`	booking.id = ?	`select * from booking_seats where booking_id = ?`
`TheatreRepository`, `ScreenRepository`	No extra methods	Only the inherited CRUD methods

11.2 Custom JPQL queries

ShowRepository.existsOverlappingShow

```
boolean existsOverlappingShow(Long screenId, LocalDate showDate, LocalTime startTime, LocalTime endTime)
```

PURPOSE

Detects a scheduling clash on one screen.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `SELECT COUNT(s) > 0 FROM Show s` counts shows on the same screen and date.
2. Condition `s.startTime < :endTime AND s.endTime > :startTime` is the standard interval-overlap test: two ranges overlap exactly when each one starts before the other ends.
3. Returns true when at least one such show exists.

NOTES

- Works only inside a single day (end must be after start). Overnight shows are not supported.

ShowRepository.findShowsByFilters

```
List<Show> findShowsByFilters(Long movieId, Long theatreId, LocalDate showDate)
```

PURPOSE

One query that supports any combination of optional filters.

HOW THE LOGIC WORKS (STEP BY STEP)

1. Joins `Show -> Screen -> Theatre` so the theatre of a show can be filtered even though Show has no direct theatre field.
2. Each condition has the shape `(:param IS NULL OR column = :param)`. If the caller passed null, the condition is always true and that filter is skipped; otherwise it must match.
3. Example: only `movieId=5` supplied -> `(false OR movie.id=5) AND (true) AND (true)` -> all shows of movie 5.

BookingRepository.findBookingsForCompletedShows

```
List<Booking> findBookingsForCompletedShows(BookingStatus status, LocalDate today, LocalTime currentTime)
```

PURPOSE

Finds bookings of the given status whose show is over.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `b.status = :status` restricts to e.g. CONFIRMED.
2. `b.show.showDate < :today` covers all earlier days.
3. `OR (b.show.showDate = :today AND b.show.endTime <= :currentTime)` covers today's shows that have already ended.
4. `b.show.x` is an implicit join from Booking to Show, written as a path in JPQL.

12. DTO Layer and Validation Rules

DTOs are plain classes carrying data between client and server. **Request DTOs** hold validation annotations; **Response DTOs** shape the JSON output and omit sensitive or internal fields. Lombok `@Getter/@Setter` (or `@Data`) plus constructors are used on all of them.

12.1 Request DTOs and their validation

DTO	Fields and rules	Used by
RegisterRequestDTO	name `@NotBlank`; email `@NotBlank @Email`; password `@NotBlank @Size(min=8)` with message "Password must contain at least 8 characters"	register
LoginRequestDTO	email `@NotBlank @Email`; password `@NotBlank`	login
UserUpdateRequestDTO	name `@NotBlank` ("Name is required"); email `@NotBlank @Email` ("Invalid email format")	updateMyProfile
MovieRequestDTO	title, genre, language, certificate `@NotBlank`; duration `@Positive`; releaseDate `@NotNull`; description optional	createMovie, updateMovieById
TheatreRequestDTO	name, address, location `@NotBlank`	addTheatre
ScreenRequestDTO	name `@NotBlank`; totalSeats `@NotNull @Positive`	addScreen
SeatRequestDTO	seatNumber `@NotBlank`; seatType `@NotNull` (REGULAR/PREMIUM)	addSeat
ShowRequestDTO	showDate `@NotNull @FutureOrPresent`; startTime, endTime, ticketPrice `@NotNull`	addShow

12.2 Response DTOs

DTO	Fields	Notes
UserResponseDTO	id, name, email, role	No password
AuthResponseDTO	token, tokenType	tokenType is always "Bearer"
MovieResponseDTO	id, title, description, genre, language, duration, releaseDate, certificate, status	
TheatreResponseDTO	id, name, address, location	
ScreenResponseDTO	id, name, totalSeats, theatreId, theatreName	Theatre fields flattened
SeatResponseDTO	id, seatNumber, seatType, screenId, screenName	Screen fields set manually in service
ShowResponseDTO	id, movieId, movieTitle, screenId, screenName, theatreId, theatreName, showDate, startTime, endTime, ticketPrice, status	Flattened from three entities
BookingResponseDTO	id, customerId, customerName, showId, totalAmount, bookingDateTime, status, bookingReference	Customer/show fields set manually
BookingSeatResponseDTO	id, bookingId, showSeatId, priceAtBooking	

13. Exception Handling

13.1 ResourceNotFoundException

A custom `RuntimeException` that carries a message. Services throw it whenever a requested row does not exist. Being unchecked, it does not need `throws` declarations and, inside a `@Transactional` method, triggers rollback.

13.2 GlobalExceptionHandler

`@RestControllerAdvice` makes this class a central place that catches exceptions thrown by any controller or service and turns them into HTTP responses. Each `@ExceptionHandler` method handles one exception type.

Handler method	Catches	Status	Response body
`handleResourceNotFound`	`ResourceNotFoundException`	404 NOT FOUND	Plain text: the exception message
`handleIllegalArgument`	`IllegalArgumentException` (all business-rule violations)	400 BAD REQUEST	Plain text: the exception message
`handleAuthendicationException`	`AuthenticationException` (bad credentials)	401 UNAUTHORIZED	Plain text: the exception message
`handleMethodArgumentNotValidException`	`MethodArgumentNotValidException` (failed `@Valid`)	400 BAD REQUEST	JSON map `{ "fieldName": "error message" }`

handleMethodArgumentNotValidException

```
public ResponseEntity<Map<String,String>>
handleMethodArgumentNotValidException(MethodArgumentNotValidException ex)
```

PURPOSE

Converts validation failures into a client-friendly field-by-field message map.

HOW THE LOGIC WORKS (STEP BY STEP)

1. `ex.getBindingResult().getFieldErrors()` returns one `FieldError` per violated rule.
2. Loop: `map.put(error.getField(), error.getDefaultMessage())`.
3. Return the map with 400. Example: `{"email":"must be a well-formed email address","password":"Password must contain at least 8 characters"}`.

NOTES

- Violations of `@Positive` on path variables are a different exception type and are not handled here.

14. Complete Endpoint Reference

Method	URL	Purpose	Success
POST	/api/v1/auth/register	Create customer account	201
POST	/api/v1/auth/login	Get JWT	200
GET	/api/v1/users/me	My profile	200
PUT	/api/v1/users/me	Update my profile	200
GET	/api/v1/users	Paged user list	200
POST	/api/v1/movies	Create movie	201
GET	/api/v1/movies	List movies	200
GET	/api/v1/movies/{id}	Get movie	200
PUT	/api/v1/movies/{id}	Update movie	200
GET	/api/v1/movies/title?title=	Search by title	200
GET	/api/v1/movies/genre?genre=	Search by genre	200
POST	/api/v1/theatres	Create theatre	201
GET	/api/v1/theatres	List theatres	200
POST	/api/v1/theatres/{theatreId}/screens	Add screen	201
POST	/api/v1/screens/{screenId}/seats	Add seat	201
GET	/api/v1/screens/{screenId}/seats	List seats of screen	200
POST	/api/v1/shows/{movieId}/{screenId}	Schedule show + generate seat map	201
GET	/api/v1/shows?movieId&theatreId&showDate	Search shows	200
GET	/api/v1/shows/{id}	Get show	200
GET	/api/v1/shows/{showId}/seats	Seat map of show	200
POST	/api/v1/bookings/users/{userId}/shows/{showId}/seats/{showSeatId}	Book a seat	201
GET	/api/v1/bookings/{bookingId}/seats	Seats of a booking	200

15. End-to-End Walkthrough

This is the order in which the methods above are used in a real scenario, and what is written to the database at each step.

Step	Who	Call	What happens in the database
1	Admin	POST /movies	Row in `movies`, status ACTIVE
2	Admin	POST /theatres	Row in `theatres`
3	Admin	POST /theatres/1/screens	Row in `screens` linked to theatre 1
4	Admin	POST /screens/1/seats (repeat per seat)	Rows in `seats` (e.g. A1 REGULAR, A2 PREMIUM)
5	Admin	POST /shows/1/1 {date, 18:00, 20:30, price 200}	Row in `shows` (SCHEDULED) + one `show_seats` row per seat: A1 = 200, A2 = 300, all AVAILABLE
6	Customer	POST /auth/register, POST /auth/login	Row in `users` (hash, role CUSTOMER); returns JWT
7	Customer	GET /shows?movieId=1	Read-only search with optional filters
8	Customer	GET /shows/1/seats	Read-only seat map; pick an AVAILABLE `showSeatId`
9	Customer	POST /bookings/users/5/shows/1/seats/2	Row in `bookings` (CONFIRMED, BK-xxxxxxxx), row in `booking_seats`, `show_seats` row 2 -> BOOKED (one transaction)
10	Customer	GET /bookings/{id}/seats	Read-only; shows the reserved seat and price
11	Scheduler	every 60 s	After 20:30 on the show day, the booking becomes COMPLETED

16. Observations: Weak Points Found While Reading the Code

These are not part of the requested explanation of the logic, but they are worth knowing before a demo, interview or deployment. Ordered by importance.

1. JWT lifetime is about 86 seconds, not one day. `EXP_TIME = 60*60*24` is in seconds but is added to `System.currentTimeMillis()`. Fix: `60L*60*24*1000`.

2. All endpoints are public. `SecurityConfiguration` uses `anyRequest().permitAll()`. Roles are never enforced, anybody can create movies or list users. The intended rules are already written (commented); enabling them is the fix. `/users/me` without a token currently throws a NullPointerException.

3. Double-booking race condition. `createBooking` checks `status == AVAILABLE` and then updates it, but nothing prevents two simultaneous requests from passing the check. Fix: add `@Version` (optimistic locking) to `ShowSeat`, or a pessimistic lock on read, and/or a unique constraint on `booking_seats.show_seat_id`.

4. User id is taken from the URL when booking. Any caller can book for any user. Better to derive the user from `Authentication`.

5. `addShow` **is not transactional.** Saving the show and saving its seats are separate commits. Adding `@Transactional` makes the pair atomic.

6. Entity exposed by ShowSeatController. Returns `ShowSeat` (with nested Show/Seat) instead of a DTO; lazy associations can cause serialisation problems and leak internal fields.

7. Defined but unused states. `BookingStatus.CANCELLED`, `ShowStatus.CANCELLED/COMPLETED`, `MovieStatus.INACTIVE` are never set. There is no cancel-booking, cancel-show or deactivate-movie feature, and no way to create an ADMIN.

8. Exceptions thrown inside the JWT filter. If a valid token refers to a deleted user, `ResourceNotFoundException` is thrown in the filter, outside `GlobalExceptionHandler` (500).

9. Unhandled validation exception type. `@Positive` on path variables raises a different exception than `MethodArgumentNotValidException`, which has no handler.

10. ModelMapper update overwrites with nulls. `updateMovieById` and `updateMyProfile` copy null fields over existing values. Validation currently prevents this for required fields, but optional ones (movie description) can be wiped.

11. Minor. Hard-coded +100 premium price; `totalSeats` not reconciled with actual seats; plain-text secrets (DB password, JWT secret) in `application.properties` should move to environment variables; `ticketPrice` accepts negative values; method name typo `handleAuthendicationException`.